

Ahsanullah University of Science & Technology
Department of Computer Science & Engineering

Real Madrid Museum — Hall of Legends

An Interactive 3D Museum built with Three.js / WebGL

Computer Graphics Lab (CSE 4204)

Final Project Report

Submitted By

Name	Student ID
Syed Abir Hossain	20220104013
Alim Bin Yeasin	20220104016

0. Project Overview

This report documents Real Madrid Museum - Hall of Legends, a browser-based, first-person interactive 3D museum built for the Computer Graphics Lab (CSE 4204) project. Three.js supplies the WebGL scene graph, geometry, cameras and model loading; hand-written GLSL shaders now shade most opaque surfaces. Visitors explore trophies, the Cristiano Ronaldo statue, animated club banners and a painting whose stadium image changes on interaction, with historical detail in a paginated magazine. The geometry, navigation and exhibit program remain intact while the principal surface materials have changed.

The ES modules remain organised under `src/`: `world/` builds the room, trophies, statue, flags, textures and custom shaders; `player/` handles first-person and eagle-eye movement; `ui/` displays the magazine. `src/main.js` coordinates the scene and interaction.

1. Problem Analysis

1.1 Project Requirements

The goal is an explorable 3D exhibition hall covering custom shaders, dynamic lighting, perspective projection, texturing, procedural animation and interactive controls. The final version deliberately uses Three.js as the 3D framework while making custom GLSL materials the main opaque rendering path. Concretely, the project set out to:

- Build a fully enclosed gallery (floor, walls, ceiling, columns and entrance) at realistic scale, using custom-shaded opaque materials.
- Model every trophy and prop as real geometry (lathed/extruded meshes) rather than using flat sprites or stock assets, so lighting and shadows behave correctly on them.
- Import, re-rig and re-pose a real skinned humanoid glTF model (Mixamo “Xbot”) plus a photo-scanned head model into a single Cristiano Ronaldo statue.
- Implement first-person WASD/mouse-look navigation with physically plausible collision against walls, columns and exhibits.
- Provide a second, eagle-eye orbit camera that shows an animated third-person avatar walking around the same room.
- Add interactive exhibits: proximity/line-of-sight detection, an on-screen prompt, and a paginated magazine UI that opens historical content per exhibit.
- Use hand-written GLSL for gallery surfaces, metals, animated cloth, stone and painting transitions rather than relying mainly on built-in surface materials.
- Keep the experience resilient — e.g. the scene must not break if the Wikimedia photo textures fail to load, or if the browser denies the Pointer Lock API.

1.2 Core Integration

Models, camera, lighting and texturing are integrated through a small set of composable builder functions that all return a `THREE.Group` plus metadata, which `main.js` then wires into one scene graph:

- **Scene assembly** — `main.js` calls `buildMuseum()`, `buildUCLExhibit()`, `buildDomesticExhibit()`, `buildStatue()`, `buildPainting()` and `buildFlags()` in sequence, each anchored to a fixed world-space position taken from `museum.anchors`, then adds their groups to a single `THREE.Scene`.

- **Texturing** — a single `TextureLibrary` class (`src/world/textures.js`) is instantiated once and injected into every builder. It draws every surface (marble floor, wall panel, ceiling coffer, carpet, wood grain, gold plaques, poster art, the club crest) procedurally on an off-screen `<canvas>`, wraps it in a `THREE.CanvasTexture`, and — once `assets/manifest.json` resolves — paints a real Wikimedia Commons photograph on top of the same canvas and calls `texture.needsUpdate = true`. This means the museum is fully textured and usable even before (or without) the network photos, and photos simply “upgrade” the same materials in place.
- **Camera** — a single `THREE.PerspectiveCamera` (72° FOV, near 0.1 / far 90) is shared by both view modes; `PlayerControls` repositions and re-aims it every frame instead of swapping cameras, and cross-fades FOV/position between the first-person eye point and the eagle-eye orbit target.
- **Lighting** - hemisphere and ambient lights wash the hall; ceiling spots, entrance sconces and exhibit lights remain in the Three.js scene. The statue has two animated spotlights. Their positions, targets, colors and strengths are also passed into its stone shader, where cone, falloff and highlight terms are calculated explicitly. A faint custom-shader beam makes the moving key light visible from the gallery floor, while the focused highlight sweeps across the figure over time.
- **Metallic appearance** - custom GLSL uses metalness, roughness-controlled specular response and a view-angle Fresnel tint on the trophies and gold trim. This replaces the former stock-material PMREM reflection description; the current main scene does not generate a PMREM environment map. The metal shader provides a lightweight reflection-like tint instead.
- **Collision integration** — rather than hand-authoring bounding boxes, any mesh/group whose `userData.solid` flag is set has its world-space `THREE.Box3` computed automatically from its geometry after the scene graph settles (`solidColliders()`), so pedestals, walls and columns can never silently drift out of sync with their visual mesh.

1.3 Technical Challenges

Several parts of the project required non-trivial applied mathematics beyond calling built-in Three.js helpers:

- **View-direction vectors from yaw/pitch** — the camera's forward vector is derived analytically each frame as $(-\sin(\text{yaw})\cos(\text{pitch}), \sin(\text{pitch}), -\cos(\text{yaw})\cos(\text{pitch}))$, a standard spherical-to-Cartesian conversion, and is reused both for movement (project onto the XZ plane) and for the interaction raycast.
- **Camera-relative movement** — keyboard input (forward/back/strafe) is rotated into world space with a 2-D rotation using $\sin(\text{yaw})/\cos(\text{yaw})$, then smoothed toward the target velocity with exponential damping `lerp(v, target, 1 - e-12·dt)` so movement never feels frame-rate dependent or twitchy.
- **Axis-separated collision resolution** — the player is moved and clamped one axis at a time (not by pushing out along the nearest face), because resolving both axes together was found to teleport the player sideways at box corners; a second correction pass then re-checks both axes, and a separate “unstuck” routine eases the player out of the single deepest-penetrated box at a capped walking speed if they ever end up inside one.
- **Dual-camera blending** — first-person and eagle-eye share one camera object. A blend factor is eased with `smoothstep(x) = x2(3-2x)` each frame, and the camera position/look-at target/FOV are all linearly interpolated between the first-person eye point and an orbiting third-person point, so the transition between modes is continuous rather than a hard cut.

- **Orbit-camera trigonometry** — the eagle-eye camera position is computed as a point on a sphere of radius `dist` around the avatar's head: $x = \text{target.x} + \sin(\text{yaw}) \cdot \cos(\text{pitch}) \cdot \text{dist}$, with the equivalent `y/z` terms, then clamped back inside the room bounds so the camera can never end up outside a wall.
- Custom cloth shader - each banner's vertex shader combines two travelling UV-space sine waves, a pinned-top slack factor and an edge factor. Time and per-banner seed uniforms animate the wave. Local wave slopes adjust the normal so highlights follow the fold. The same shader module also provides shared surface, metallic and stone fragments, a painting texture blend and a translucent statue spotlight beam.
- **Skeletal re-posing via quaternions** — the statue's body is a rigged Mixamo model whose skeleton is driven by two small helper functions, `aim()` and `twist()`, that read a bone's and its child's world-space position from their `matrixWorld`, compute the rotation needed to align the bone with a target direction, and premultiply it onto the bone's local quaternion — effectively solving a per-joint orientation problem to bend the default T/A-pose into the statue's “Siuu” landing stance.
- **Procedural kit geometry on a moving skeleton** — the statue's shorts, sleeves and boots are generated as tube/ribbon meshes whose control points are the interpolated (`lerp`) world-space positions between pairs of joints, so the carved “clothing” conforms to the re-posed skeleton instead of being modelled once in a fixed pose.
- **Lathed and swept trophy profiles** — each trophy body is a `THREE.LatheGeometry` revolved from a hand-tuned 2-D radius/height profile matched to the real trophy's silhouette, while the handles are flat metal ribbons extruded along a `CatmullRomCurve3` using the curve's normal/binormal frame, so they read as flat scrollwork rather than round tubes.
- **Frame-rate independent simulation** — the render loop measures true elapsed time with `THREE.Clock` but clamps any delta above 120 ms (a tab switch, a GC pause, texture decoding) down to a fixed 1/60 s step, preventing the player from being catapulted across the room after the browser stalls.

2. User Interaction

2.1 Key Interaction

Keyboard state is tracked in a Set updated on keydown/keyup, decoupled from any single “once-per-press” event so multiple keys can be held simultaneously:

- `W / A / S / D` or the arrow keys drive forward/back/strafe. Held keys are normalised into a (`fwd`, `strafe`) pair each frame, rotated by the player's current yaw into world space, and normalised so diagonal movement is not faster than cardinal movement.
- `Shift` enables sprinting, raising the target speed from 4.4 to 7.2 world-units/second.
- `E` (or a left click while looking at an exhibit) activates whichever exhibit the player is currently facing/near — opening the UCL, Ballon d’Or or Domestic Honours magazine, or toggling the Bernabéu painting between its old and new photographs.
- `V` (or the on-screen button) switches between first-person and eagle-eye camera modes.
- `Esc` releases the pointer lock and hands control back to the OS cursor; releasing the lock never disables the controls — drag-to-look and the movement keys keep working regardless.

- Inside the magazine overlay, the left/right arrow keys (or `h/l`) turn pages and `E/Esc` closes it; these keys are consumed by the overlay so the same keypress can never “leak through” and immediately re-trigger the exhibit behind it.

2.2 Mouse Interaction

Looking around is implemented with two cooperating input paths so the player is never left with an unresponsive camera:

- **Pointer-lock look** — once the canvas requests the browser's Pointer Lock API, raw `mousemove` deltas rotate yaw and pitch directly, with the very first post-lock event discarded (it otherwise carries a large jump from wherever the OS cursor happened to be) and every delta clamped to ± 160 px so a sudden spike cannot whip the view.
- **Drag-to-look fallback** — whenever the lock is not held (denied by the browser, or the player pressed `Esc`), a pointer-down + pointer-move + pointer-up sequence is tracked instead: dragging turns the camera by the same yaw/pitch update, while a short, low-movement drag is reinterpreted as a plain click so short taps still trigger exhibit interaction.
- **Click-to-interact** — a left click while a target is in range (and not a drag) calls the same `activate()` path as pressing `E`.
- **Mouse-wheel zoom** — in eagle-eye mode only, the wheel adjusts the orbit distance (clamped between 3.2 and 16 units) and is eased toward the target distance every frame instead of snapping.

3. Design and Aesthetic Aspects

3.1 Visual Elements

Most visible opaque surfaces now use custom GLSL ShaderMaterial variants. Three.js still manages their meshes, transforms, textures and rendering; the entrance glass keeps `MeshPhysicalMaterial` for its transparent appearance. The main visual elements are:

- **Procedural + photographic textures** - `TextureLibrary` creates canvas textures for the terrazzo floor, quiet plaster walls, ceiling, wood, plaques and banners. Local stadium, trophy and player photographs are applied to exhibit art. The custom gallery and metal shaders sample these textures and combine them with diffuse/specular terms; roughness and metalness uniforms keep the surfaces visually distinct. The canvases provide an immediate procedural fallback before optional image assets load, so missing photographs do not leave exhibits blank. Gold tinting keeps real photos within the warm museum palette.
- **Lighting model** - the shared fragment shader evaluates directional key/fill response, roughness-controlled highlights and metallic Fresnel tint. The stone variant adds two focused, moving spot calculations from uniforms, while a translucent cone follows the key light. Three.js still supplies hall/exhibit light objects and renderer-side PCF soft shadows; ACES filmic tone mapping (exposure 1.12) keeps bright accents under control. Because custom ShaderMaterial does not automatically inherit standard-material light response, the monument's moving light is evaluated directly in GLSL rather than merely positioning a Three.js spotlight.
- **Colour and atmosphere** — the scene background and `THREE.Fog` are set to a near-black navy (`#0a0c12`), so distant walls fade softly instead of hard-clipping at the far plane, reinforcing the sense of a large, dim exhibition hall punctuated by warm spotlighting.
- **Cloth banners** — sixteen club banners (in alternating white and navy “way kit” colourways) hang from procedural gold rods around the hall, each displaying a hand-drawn Real Madrid crest (interlaced RMCF

monogram, gold crown, mulberry band) and animate continuously via the custom shader described in §1.3/§4.2.

- The Cristiano Ronaldo statue - the re-posed figure is covered by a custom weathered-stone shader using granular detail and two animated spotlight calculations. A faint moving beam and changing highlights across the figure make the focused gallery lighting visible. The skinned body, scanned head and generated kit remain separate geometry under the same sculptural treatment.

3.2 Snapshots

The following screenshots, captured directly from the running application, illustrate the main features described above.

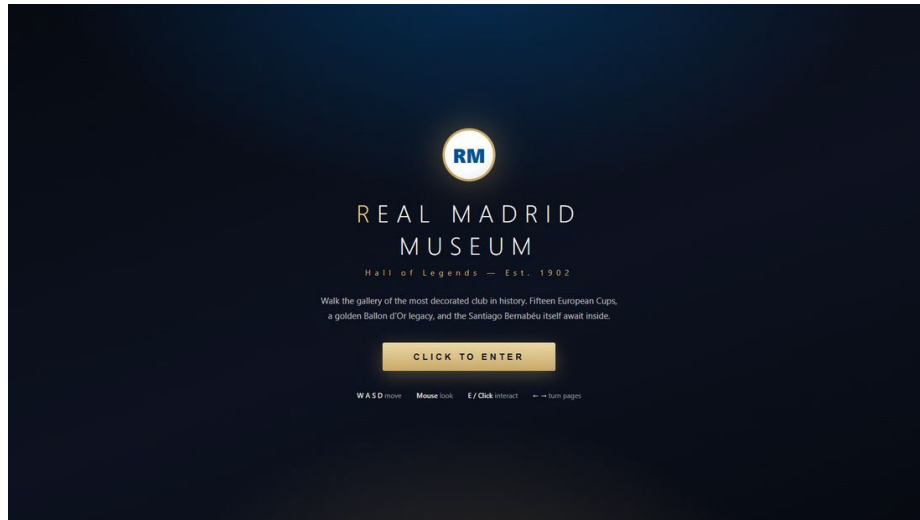


Fig. 1 — Splash / entry screen with control summary.

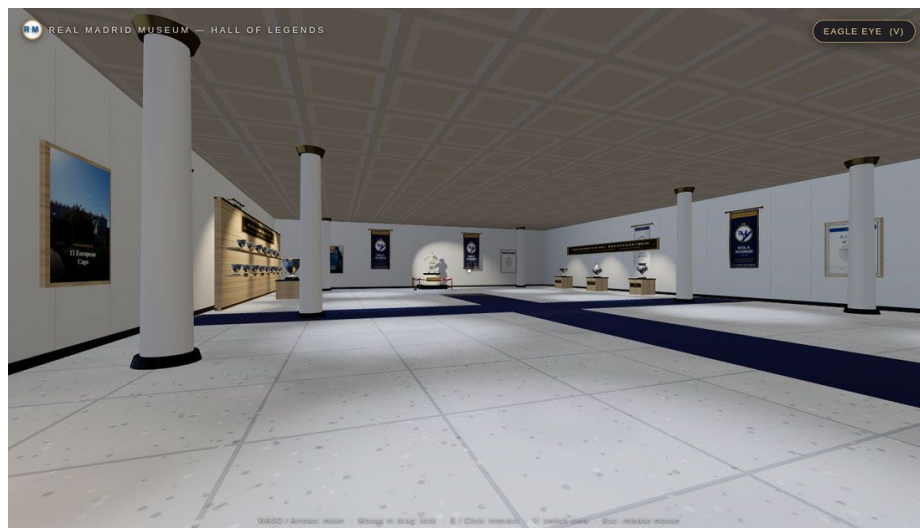


Fig. 2 — Wide eagle-eye view of the Hall of Legends: entrance, UCL wall, columns, carpet cross and posters.



Fig. 3 — UCL Trophy Wall exhibit: fifteen lathed European Cup replicas with year plaques.



Fig. 4 — Domestic Honours exhibit: the Copa del Rey trophy on its wooden pedestal.

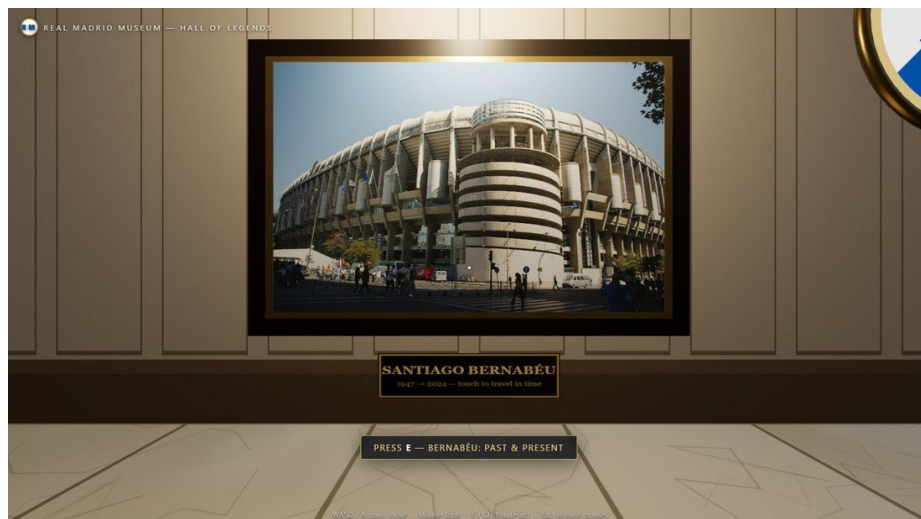


Fig. 5 — The Santiago Bernabéu painting exhibit, lit by its own picture light.

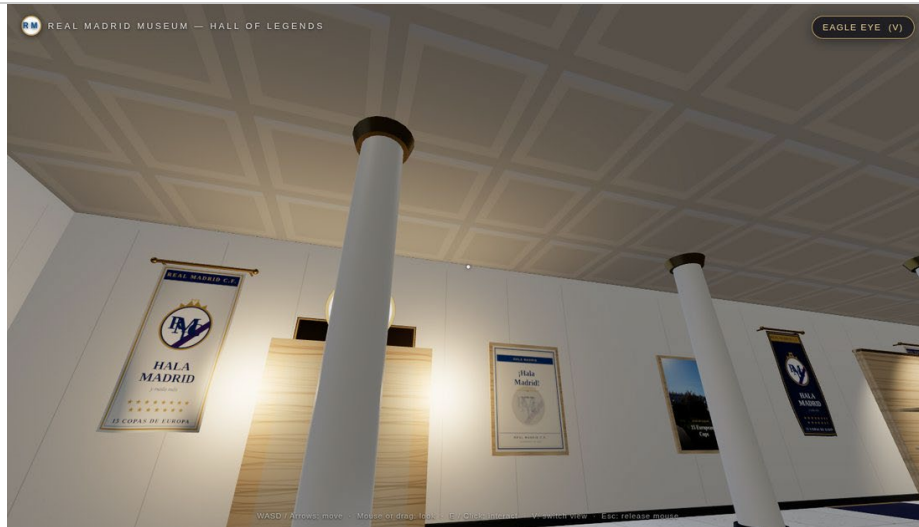


Fig. 6 — Animated cloth banners along the north wall, driven by the custom vertex shader.

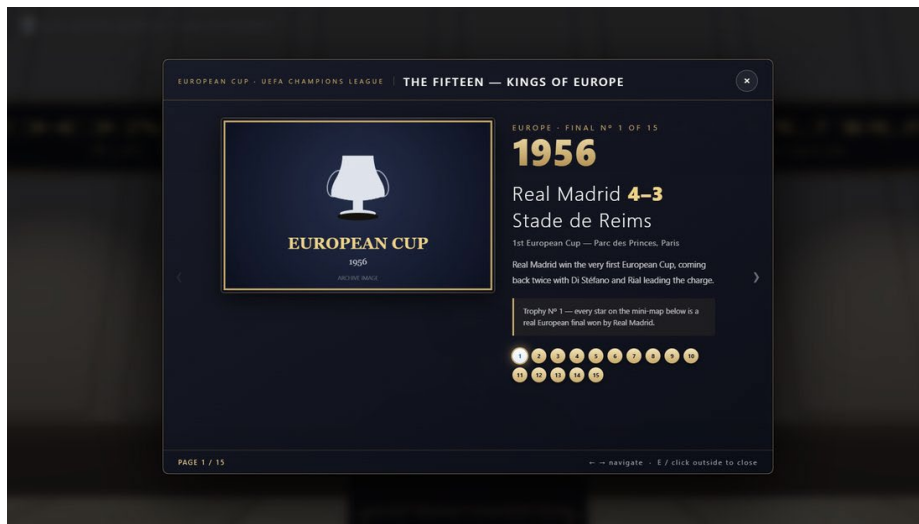


Fig. 7 — The paginated magazine overlay opened on the UCL Trophy Room (Page 1 of 15).

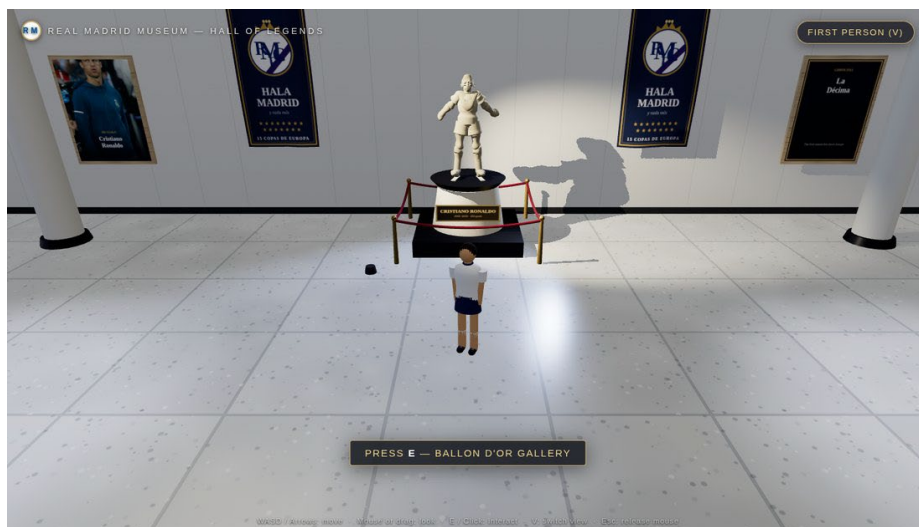


Fig. 8 — Eagle-eye camera showing the animated visitor avatar approaching the Cristiano Ronaldo statue.

4. Code Implementation

4.1 Software Platform

The project is a small, dependency-light static web application:

- **Languages** — HTML5, CSS3, and modern JavaScript (ES modules, top-level `await`).
- **Graphics** - Three.js r0.182 provides scene graph, cameras, geometry, lights and shadow mapping; GLTFLoader imports the statue models. Hand-written GLSL in `src/world/shaders.js` uses `ShaderMaterial` for the principal opaque surfaces, cloth animation, painting blend and light beam.
- **Runtime / tooling** — a minimal Node.js static file server (`server.js`, started with `npm start`) serves the app over HTTP so ES-module imports and texture/model fetches are not blocked by the `file://` origin restriction; `package.json` declares Three.js as the sole runtime dependency.
- **Assets** — Real photographs are sourced from Wikimedia Commons via `tools/download_assets.py` into `assets/img/`, indexed by `assets/manifest.json`; the statue's base body/head come from the standard three.js sample glTF assets (`Xbot.glb`, `LeePerrySmith.glb`) under `assets/models/`.
- **Testing / QA** - `tools/selftest.js` and `tools/collisiontest.js` exercise interaction and collision through `window.__museum.step()` via Playwright. `tools/verify_required_features.js` checks camera movement, the sweeping statue spotlight and the painting's old/new texture blend; `tools/shots.js` captures the gallery images used in section 3.2.

4.2 Custom Shaders

The project now uses hand-written GLSL beyond the banners. `src/world/shaders.js` defines shared `ShaderMaterial`-based surface, metal and stone materials, plus animated cloth, painting blend and a visible spotlight beam. Three.js remains responsible for geometry, cameras and the WebGL render loop. The banner vertex shader still illustrates the core animation technique:

- **Vertex displacement** — a GLSL function `wave(uv)` sums two travelling sine waves of different spatial frequency, phase speed and per-banner seed, then scales the result by a “slack” falloff `pow(1.0 - uv.y, 1.35)` (nearly rigid at the top rod, loosest at the hem) and an “edge” falloff based on horizontal distance from centre — producing a believable hanging-cloth silhouette rather than a uniform ripple.
- **Analytic normal response** - local wave slopes are computed in the vertex shader and used to adjust the cloth normal, so the lighting changes with the moving fold instead of following only the flat undeformed surface.
- **Uniform-driven animation** — a single `uTime` uniform (advanced once per frame from the main render loop) drives every banner; a per-instance `uSeed` uniform offsets the wave phase per banner so the sixteen flags do not all ripple in lock-step.
- **Hem sway** - the displaced vertex also shifts slightly downward as the wave grows, adding secondary motion to the cloth rather than a uniform back-and-forth ripple.

`GallerySurfaceShader` and `MuseumMetalShader` handle the hall and trophies with texture, roughness, metalness, specular and fog terms. `StoneStatueShader` receives moving spot uniforms; `PaintingBlendShader` mixes historic and modern photographs by `uBlend`. The entrance glass retains `MeshPhysicalMaterial`. This is custom shading for most opaque visuals, not a complete replacement of Three.js or a full physically based renderer.

4.3 Project Feature Table

#	Feature	Status
---	---------	--------

1	3D scene: room, walls, ceiling, columns, carpet, entrance doors	Implemented
2	Procedural and photo textures on custom GLSL gallery surfaces	Implemented
3	Perspective camera with two modes (first-person / eagle-eye orbit)	Implemented
4	Keyboard movement (WASD / Arrow keys) with camera-relative direction	Implemented
5	Mouse-look via Pointer Lock API, with click-and-drag fallback	Implemented
6	AABB collision detection & response (walls, columns, exhibits, statue)	Implemented
7	Dynamic lighting: hemisphere, ambient, spot and point lights + shadow maps	Implemented
8	Custom GLSL shading across the museum	Implemented
9	Procedurally lathed/extruded trophy models (UCL, LaLiga, Copa, Supercopa)	Implemented
10	Cristiano Ronaldo statue built and re-posed from real glTF rigs	Implemented
11	Interactive exhibits (press E) opening a paged magazine overlay	Implemented
12	Historical data pages (15 UCL finals, Ballon d'Or, domestic honours)	Implemented
13	Animated eagle-eye visitor avatar with walk-cycle & turn-to-face	Implemented
14	Shader-blended old/new Bernabeu painting textures	Implemented
15	Moving statue spotlight and visible light beam	Implemented
16	ACES Filmic tone mapping and scene fog for depth cueing	Implemented
17	Window-resize responsive rendering	Implemented
18	Automated headless tests & screenshot capture tooling (Playwright)	Implemented
19	Sprint (Shift) movement and footstep view-bob	Implemented
20	Mobile / touch-specific control scheme	Not Implemented

Table 01: Project Feature Table

5. Contribution

Both members collaborated closely throughout the project (shared design decisions, joint debugging and testing), with primary ownership split roughly as follows:

Syed Abir Hossain (20220104013)

- Core rendering setup: renderer, scene and camera configuration, tone mapping, fog and shared gallery lighting.
- Player systems: first-person / eagle-eye camera blending, pointer-lock and drag-to-look input, keyboard movement, and the axis-separated collision detection & resolution logic.
- Lighting design across the hall and per-exhibit accent spotlights, and the base TextureLibrary (procedural floor/wall/ceiling/wood/marble textures and photo-overlay pipeline).
- The custom cloth vertex shader for the waving banners.

Alim Bin Yeasin (20220104016)

- Exhibit and trophy modelling: the lathed/extruded UCL, LaLiga, Copa del Rey and Supercopa trophies, pedestals and wall displays.
- The Cristiano Ronaldo statue: glTF import, skeletal re-posing (aim/twist helpers), procedural kit geometry, and the stone material/texture treatment.
- UI/UX: the magazine overlay (open/close, page navigation) and the historical content pages (UCL finals, Ballon d'Or, domestic honours data).
- Testing and asset tooling: the headless self-test and collision-test scripts, screenshot automation, and asset-download utilities.